Datalog Reasoning

Simon Frost

Overview

Datalog is a declarative logic programming language that operates over relations (sets of tuples). RDFLib.jl includes a Datalog reasoner that uses semi-naive bottom-up evaluation to compute the logical closure of an RDF graph augmented with N3 rules. It is designed for workloads consisting of pure Datalog rules — conjunctive queries with no built-in predicates — where set-at-a-time evaluation with hash joins can outperform the general-purpose N3 reasoner.

```
using RDFLib
```

Writing Rules in N3

Rules are written in Notation3 syntax using `{ antecedent } => { consequent }`. The antecedent (body) is a conjunction of triple patterns; the consequent (head) is one or more triple patterns to derive when the body matches.

```
n3 = """
@prefix : <http://example.org/> .

:alice :type :Person .
:bob :type :Person .

{ ?x :type :Person } => { ?x :is :human } .
"""

g = parse_rdf(n3, N3Format())
result = datalog_reason(g)

for t in triples(result, (nothing, URIRef("http://example.org/is"), nothing))
    println(t.subject, " is ", t.object)
end
```

```
URIRef("http://example.org/alice") is URIRef("http://example.org/human")
URIRef("http://example.org/bob") is URIRef("http://example.org/human")
```

Variables (prefixed with `?`) in the body are matched against data triples. When all body atoms are satisfied by a consistent binding, the head atoms are instantiated and added to the graph. This process repeats until no new triples are derived (fixpoint).

Transitive Closure

The most common Datalog pattern is transitive closure — deriving all reachable pairs from a relation:

```
n3 = """
@prefix : <http://example.org/> .

:a :p :b .
:b :p :c .
:c :p :d .

{ ?x :p ?y . ?y :p ?z } => { ?x :p ?z } .
"""

g = parse_rdf(n3, N3Format())
result = datalog_reason(g)

p = URIRef("http://example.org/p")
for t in triples(result, (nothing, p, nothing))
    println("  ", t.subject, " :p ", t.object)
end
```

```
  URIRef("http://example.org/a") :p URIRef("http://example.org/b")
  URIRef("http://example.org/b") :p URIRef("http://example.org/c")
  URIRef("http://example.org/c") :p URIRef("http://example.org/d")
  URIRef("http://example.org/a") :p URIRef("http://example.org/c")
  URIRef("http://example.org/b") :p URIRef("http://example.org/d")
  URIRef("http://example.org/a") :p URIRef("http://example.org/d")
```

Starting from three direct edges, the reasoner derives all transitive connections: `:a :p :c` (via `:b`), `:b :p :d` (via `:c`), and `:a :p :d` (via the chain `:a→:b→:c→:d`).

Multi-Predicate Reasoning: Family Relationships

Rules can join across different predicates and derive new relationships. Here we infer grandparent and sibling relations from parent data:

```
n3 = """
@prefix : <http://example.org/> .

:alice :parent :bob .
:alice :parent :carol .
:bob :parent :dave .
:bob :parent :eve .
:carol :parent :frank .

{ ?x :parent ?y . ?y :parent ?z } => { ?x :grandparent ?z } .
{ ?p :parent ?x . ?p :parent ?y } => { ?x :sibling ?y } .
"""

g = parse_rdf(n3, N3Format())
result = datalog_reason(g)

gp = URIRef("http://example.org/grandparent")
sib = URIRef("http://example.org/sibling")

println("Grandchildren of Alice:")
for t in triples(result, (URIRef("http://example.org/alice"), gp, nothing))
    println("  ", t.object)
end

println("\nSibling pairs:")
seen = Set{Tuple{String,String}}()
for t in triples(result, (nothing, sib, nothing))
    a, b = string(t.subject), string(t.object)
    if a != b
        pair = (min(a, b), max(a, b))
        if pair  seen
            push!(seen, pair)
            println("  ", t.subject, "  ", t.object)
        end
    end
end
```

```
Grandchildren of Alice:
```

```
  URIRef("http://example.org/eve")
  URIRef("http://example.org/dave")
  URIRef("http://example.org/frank")

Sibling pairs:
  URIRef("http://example.org/bob")   URIRef("http://example.org/carol")
  URIRef("http://example.org/dave")   URIRef("http://example.org/eve")


Grandchildren of Alice:
  URIRef("http://example.org/dave")
  URIRef("http://example.org/eve")
  URIRef("http://example.org/frank")

Sibling pairs:
  URIRef("http://example.org/bob")   URIRef("http://example.org/carol")
  URIRef("http://example.org/dave")   URIRef("http://example.org/eve")
```

Note that the sibling rule `{ ?p :parent ?x . ?p :parent ?y } => { ?x :sibling ?y }` also derives self-siblings (`:bob :sibling :bob`) since pure Datalog has no built-in inequality. Above we filter those out when printing. If you need inequality, use the full N3 reasoner which supports `log:notEqualTo`.

RDFS-Style Reasoning

Datalog is well suited for RDFS-style inference. Two simple rules implement transitive subclass reasoning and type propagation:

```
n3 = """
@prefix : <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:Cat rdfs:subClassOf :Feline .
:Feline rdfs:subClassOf :Mammal .
:Mammal rdfs:subClassOf :Animal .

:whiskers rdf:type :Cat .

{ ?c1 rdfs:subClassOf ?c2 . ?c2 rdfs:subClassOf ?c3 }
    => { ?c1 rdfs:subClassOf ?c3 } .
```

```
{ ?x rdf:type ?c . ?c rdfs:subClassOf ?d }
    => { ?x rdf:type ?d } .
"""

g = parse_rdf(n3, N3Format())
result = datalog_reason(g)

whiskers = URIRef("http://example.org/whiskers")
rdf_type = URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type")

println("Whiskers is a:")
for t in triples(result, (whiskers, rdf_type, nothing))
    println("  ", t.object)
end
```

```
Whiskers is a:
  URIRef("http://example.org/Animal")
  URIRef("http://example.org/Cat")
  URIRef("http://example.org/Feline")
  URIRef("http://example.org/Mammal")

Whiskers is a:
  URIRef("http://example.org/Cat")
  URIRef("http://example.org/Mammal")
  URIRef("http://example.org/Feline")
  URIRef("http://example.org/Animal")
```

The subclass transitivity rule first derives `:Cat rdfs:subClassOf :Mammal`, `:Cat rdfs:subClassOf :Animal`, and `:Feline rdfs:subClassOf :Animal`. The type propagation rule then uses all subclass relationships to infer that `:whiskers` is a member of every superclass.

Multiple Head Atoms

A single rule can derive multiple triples at once by placing several triple patterns in the consequent:

```
n3 = """
@prefix : <http://example.org/> .
```

```
:alice :parent :bob .
:bob :parent :carol .

{ ?x :parent ?y . ?y :parent ?z }
    => { ?x :grandparent ?z . ?z :grandchild ?x } .
"""

g = parse_rdf(n3, N3Format())
result = datalog_reason(g)

alice = URIRef("http://example.org/alice")
carol = URIRef("http://example.org/carol")
gp = URIRef("http://example.org/grandparent")
gc = URIRef("http://example.org/grandchild")

println("Grandparent: ", Triple(alice, gp, carol) in result)
println("Grandchild:  ", Triple(carol, gc, alice) in result)
```

```
Grandparent: true
Grandchild:  true
```

```
Grandparent: true
Grandchild:  true
```

Both the forward (`:grandparent`) and inverse (`:grandchild`) relationships are derived from a single rule application.

Multiple Interacting Rules

Rules can interact — the conclusions of one rule feed the body of another. Here, transitive subclass closure feeds type inference:

```
n3 = """
@prefix : <http://example.org/> .

:a :subClassOf :b .
:b :subClassOf :c .
:x :type :a .

{ ?c1 :subClassOf ?c2 . ?c2 :subClassOf ?c3 }
```

```
    => { ?c1 :subClassOf ?c3 } .

{ ?x :type ?c . ?c :subClassOf ?d }
    => { ?x :type ?d } .
"""

g = parse_rdf(n3, N3Format())
result = datalog_reason(g)

x = URIRef("http://example.org/x")
tp = URIRef("http://example.org/type")

println("x has types:")
for t in triples(result, (x, tp, nothing))
    println("  ", t.object)
end
```

```
x has types:
  URIRef("http://example.org/c")
  URIRef("http://example.org/a")
  URIRef("http://example.org/b")


x has types:
  URIRef("http://example.org/a")
  URIRef("http://example.org/c")
  URIRef("http://example.org/b")
```

The first rule derives `:a :subClassOf :c`, which the second rule uses together with `:x :type :a` to derive `:x :type :c`.

Data Without Rules

When no rules are present, `datalog_reason` returns the original data unchanged:

```
n3 = """
@prefix : <http://example.org/> .
:a :p :b .
:c :p :d .
"""
```

```
g = parse_rdf(n3, N3Format())
result = datalog_reason(g)
println("Triples: ", length(result))
```

```
Triples: 2
```

Comparison with the N3 Reasoner

RDFLib.jl provides two reasoning engines. Choose based on your needs:

Feature	`datalog_reason`	`reason` (N3)
Evaluation	Semi-naive bottom-up	Euler Abstract Machine (backward chaining)
Rule scope	Pure Datalog (conjunctive body, no built-ins)	Full N3 (built-ins, negation, backward chaining)
Built-in predicates	None	100+ (math, string, list, crypto, log)
Backward chaining (`<=`)	Not supported	Supported
Inequality (`log:notEqualTo`)	Not supported	Supported
Multiple head atoms	Supported	Supported
Optimal for	Large-scale relational inference	Rules needing built-in computations

Both reasoners produce identical results for pure Datalog rules:

```
n3 = """
@prefix : <http://example.org/> .
:a :p :b .
:b :p :c .
:c :p :d .
:d :p :e .
{ ?x :p ?y . ?y :p ?z } => { ?x :p ?z } .
"""

g = parse_rdf(n3, N3Format())

dl_result = datalog_reason(g)
n3_result = reason(g)
```

```
p = URIRef("http://example.org/p")
dl_triples = Set(t for t in triples(dl_result) if t.predicate == p)
n3_triples = Set(t for t in triples(n3_result) if t.predicate == p)

println("Results match: ", dl_triples == n3_triples)
println("Triples derived: ", length(dl_triples))
```

```
Results match: true
Triples derived: 10
```

```
Results match: true
Triples derived: 10
```

When to use `datalog_reason`: Your rules are pure Datalog — no built-in predicates, no backward chaining, no inequality. You want straightforward relational inference (transitive closure, type propagation, classification).

When to use `reason`: You need built-in predicates (`math:sum`, `string:length`, `log:notEqualTo`), backward chaining (`<=`), meta-reasoning (`log:conclusion`), or list/formula manipulation.

Performance Characteristics

The Datalog reasoner uses several optimizations:

- **Semi-naive evaluation:** Each iteration only considers newly derived facts (delta tuples), avoiding redundant work from re-processing previously known facts.
- **Integer encoding:** RDF terms are mapped to compact integer IDs before evaluation. All pattern matching operates on integers rather than string comparisons.
- **Hash joins:** Multi-body rules use hash-based indices (SPO, POS, PSO) to efficiently join body atoms on shared variables.
- **Body reordering:** The reasoner automatically reorders body atoms using a greedy algorithm that maximizes the number of bound variables at each join step.

The `max_iterations` parameter (default 1000) bounds the evaluation loop. For graphs where the fixpoint requires more iterations, increase it:

```
result = datalog_reason(g; max_iterations=5000)
```

```
RDFGraph (10 triples)
```

For large transitive chains, the number of derived triples grows quadratically — a chain of n edges produces $n(n+1)/2$ total reachable pairs:

```
n3_lines = ["@prefix : <http://example.org/> ."]
for i in 0:49
    push!(n3_lines, ":n$i :p :n$(i+1) .")
end
push!(n3_lines, "{ ?x :p ?y . ?y :p ?z } => { ?x :p ?z } .")

g = parse_rdf(join(n3_lines, "\n"), N3Format())
result = datalog_reason(g)

p = URIRef("http://example.org/p")
p_count = count(t -> t.predicate == p, triples(result))
println("50-element chain → $p_count :p triples (expected $(50*51÷2))")
```

```
50-element chain → 1275 :p triples (expected 1275)
```